

OI 板子

目录

1. OI 模板	3	5.2. 去重	7
2. __int128	3	5.3. 离散化	7
2.1. C 风格	3	5.4. 中位数	7
2.2. C++ 风格	4	5.5. 逆序数	7
3. bit	4	6. 倍增	8
3.1. qpow(a,x,MOD)	4	6.1. 倍增分段	8
3.2. lowbit(x)	4	6.2. ST 算法	9
4. 二分	5	7. 搜索	9
4.1. 库函数	5	7.1. BFS	9
4.2. 整数集合上的二分	5	7.2. DFS	9
4.3. 实数域上的二分	5	7.3. ID-DFS	9
5. 排序	6	7.4. A*	10
5.1. 库函数	6	7.5. ID-A*	10
		8. 双指针	11
		8.1. 相向指针	11
		8.2. 快慢指针	11
		8.3. 滑动窗口	11

8.4. 分离指针	11	12.6. 网络流	29
9. 进阶数据结构	11	12.7. 连通性	32
9.1. 并查集	11	13. 字符串	37
9.2. 线段树	12	13.1. KMP 模式匹配	37
9.3. 树状数组	14	13.2. Trie 字典树	38
10. 数论	15	13.3. SA	38
10.1. 质数	15	13.4. SAM	39
10.2. gcd	16	14. 随机化	40
10.3. 因数	16	14.1. 随机函数	40
11. 动态规划	17	14.2. 爬山与退火	41
11.1. 背包	17	15. 离线算法	41
12. 图论	19	15.1. 整体二分	41
12.1. 存储结构	19	15.2. 莫队算法	47
12.2. 最短路	21		
12.3. 树	23		
12.4. 生成树	26		
12.5. 二分图	28		

1. OI 模板

```
#include <iostream>
#include <iomanip>
#include <array>
#include <vector>
#include <queue>
#include <stack>
#include <map>
#include <set>
#include <unordered_map>
#include <unordered_set>
#include <bitset>
#include <string>
#include <numeric>
#include <algorithm>
#include <cmath>
#include <climits>
#include <cstring>
#include <cstdint>
using namespace std;
#define int long long

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    return 0;
}
```

2. __int128

2.1. C 风格

```
// C 风格
__int128 read() {
    __int128 x = 0, f = 1;
    char ch = getchar();
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        ch = getchar();
    }
    return x * f;
}

void print(__int128 x) {
    if (x < 0) {
        putchar('-');
        x = -x;
    }
    if (x > 9) {
        print(x / 10);
    }
    putchar(x % 10 + '0');
}
```

2.2. C++风格

```
// C++风格
istream& operator>>(istream& is, __int128& a) {
    __int128 x = 0, f = 1;
    char ch;
    is.get(ch);
    while (ch < '0' || ch > '9') {
        if (ch == '-')
            f = -1;
        is.get(ch);
    }
    while (ch >= '0' && ch <= '9') {
        x = x * 10 + ch - '0';
        is.get(ch);
    }
    a = x * f;
    return is;
}

ostream& operator<<(ostream& os, __int128 x) {
    if (x < 0) {
        os << '-';
        x = -x;
    }
    if (x > 9) {
        os << __int128(x / 10);
    }
    os << char(x % 10 + '0');
    return os;
}
```

3. bit

3.1. qpow(a,x,MOD)

```
int qpow(int a, int b, int MOD) {
    int ret = 1;
    while (b) {
        if (b & 1) {
            ret = ret * a % MOD;
        }
        a = a * a % MOD;
        b >>= 1;
    }
    return ret;
}
```

3.2. lowbit(x)

```
int lowbit(int x) {
    return x & (-x);
}
```

4. 二分

4.1. 库函数

```
equal_range();
lower_bound();
upper_bound();
binary_search();
equal_range=[lower_bound,upper_bound)
binary_search=is_valid(equal_range)
```

4.2. 整数集合上的二分

```
/**
 * @name lower_bound
 * @brief 返回第一个大于等于 x 的下标
 * @note 按照左闭右开的规范
 */
int lower_bound(const vector<int>& a, int l, int r, int x) {
    while (l < r) {
        // mid取不到 r
        // 下面三句话配套出现
        int mid = (l + r) >> 1;
        if (a[mid] >= x) {
            r = mid;
        }
        else {
            l = mid + 1;
        }
    }
}
```

```
    }
}
// 二分终止条件: l==r
return l;
}
/**
 * @name
 * @brief 返回最后一个小于等于 x 的下标
 * @note 按照左闭右开的规范
 */
int bound(const vector<int>& a, int l, int r, int x) {
    while (l < r) {
        // mid取不到 l
        // 下面三句话配套出现
        int mid = (l + r + 1) >> 1;
        if (a[mid] <= x) {
            l = mid;
        }
        else {
            r = mid - 1;
        }
    }
    // 二分终止条件: l==r
    return l;
}
```

4.3. 实数域上的二分

```
bool calc(double x) {
    return x >= 114514.1919810;
}
```

```
double bound(double l, double r) {  
    while (l + 1e-5 < r) {  
        double mid = (l + r) / 2;  
        if (calc(mid)) {  
            r = mid;  
        }  
        else {  
            l = mid;  
        }  
    }  
    return l;  
}
```

5. 排序

5.1. 库函数

5.1.1. 快速排序

```
sort(iterator begin, iterator end [, __Function cmp]);
```

5.1.2. 稳定排序

```
stable_sort(iterator begin, iterator end [, __Function cmp]);
```

5.1.3. 前`mid-begin`个都有序

```
partial_sort(iterator begin, iterator mid, iterator end [, __Function cmp]);
```

5.1.4. 第`mid-begin`个位置正确

```
nth_element(iterator begin, iterator nth, iterator end [, __Function cmp]);
```

5.1.5. 判断有序

```
is_sorted(iterator begin, iterator end [, __Function cmp]);
```

5.2. 去重

```
sorted.erase(std::unique(sorted.begin(), sorted.end()), sorted.end());
```

5.3. 离散化

```
vector<int> discrete(vector<int> a) {  
    if (a.empty()) {  
        return {};  
    }  
    sort(a.begin(), a.end());  
    a.erase(std::unique(a.begin(), a.end()), a.end());  
    vector<int> r;  
    r.push_back(a.front());  
    int last = a.front();  
    for (auto x : a) {  
        if (last != x) {  
            r.push_back(x);  
            last = x;  
        }  
    }  
    return r;  
}
```

5.4. 中位数

中位数到所有点的距离之和最小

5.5. 逆序数

```
int tmp[114514];  
int merge(int a[], int l, int r) { // 左闭右开  
    int mid = (l + r) / 2;  
    int cnt = 0;  
    int i = l, j = mid;  
    int k = l;  
    while (i < mid && j < r) {  
        if (a[i] <= a[j]) {  
            tmp[k++] = a[i++];  
        }  
        else {  
            tmp[k++] = a[j++];  
            cnt += mid - i;  
        }  
    }  
    while (i < mid) {  
        tmp[k++] = a[i++];  
    }  
    while (j < r) {  
        tmp[k++] = a[j++];  
    }  
    for (k = l; k < r; ++k) {  
        a[k] = tmp[k];  
    }  
    return cnt;  
}  
int merge_sort(int a[], int l, int r) {  
    if (r - l <= 1) {
```

```

        return 0;
    }
    int cnt = 0;
    int mid = (l + r) / 2;
    cnt += merge_sort(a, l, mid);
    cnt += merge_sort(a, mid, r);
    cnt += merge(a, l, r);
    return cnt;
}

```

6. 倍增

6.1. 倍增分段

```

/**
 * @brief 倍增分段
 * @note 左闭右开
 */
void beizeng(vector<int> a) {
    int n = a.size();

    for (int l = 0, r = 0; l < n;) {
        int p = 1;

        while (p) {
            // 左闭右开
            if (r + p <= n && check(l, r + p)) {
                r += p;
                p <<= 1;
            }
            else {
                p >>= 1;
            }
        }

        push_ans(l, r);
        l = r;
    }
}

```

6.2. ST 算法

```
/**
 * @brief ST 算法
 * @details f[i][j]记录[i,i+2^j)内的最大值
 * @details 先进行 O(nlogn)的预处理，之后可以进行 O(1)的在线回答
 */
const int N = 1e5 + 5;
const int LOGN = 20;
int n;
int a[N];
int f[N][LOGN];
void st_prework() {
    for (int i = 1; i <= n; ++i) {
        f[i][0] = a[i];
    }
    int t = log2(n) + 1;
    for (int j = 1; j < t; ++j) {
        for (int i = 1; i < n - (1 << j); ++i) {
            f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]);
        }
    }
}
int st_query(int l, int r) {
    int k = log2(r - l + 1);
    return max(f[l][k], f[r - (1 << k) + 1][k]);
}
```

7. 搜索

7.1. BFS

```
void bfs(int x) {
    queue<int> q;
    q.push(x);
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if () {
            q.push(v);
        }
    }
}
```

7.2. DFS

```
void dfs(int u) {
    if () {
        return;
    }
    dfs(v);
}
```

7.3. ID-DFS

```

bool iddfs(int u, int d) {
    if (!d) {
        return false;
    }

    iddfs(v, d - 1);
    return true;
}

void solve() {
    for (int deep = 0;; ++deep) {
        if (iddfs(s, deep)) {
            break;
        }
    }
}

```

7.4. A*

```

/**
 * @brief A*
 * @note Dijkstra 版 A*, dis[v]=g[v]+h[v]
 */
struct Edge {
    int v, w;
};
struct Node { /// Dijkstra 节点
    int dis, u;
    bool operator>(const Node& a) const {
        return dis > a.dis;
    }
};

```

```

vector<Edge> edge[N];
int g[N];
bool vis[N];
priority_queue<Node, vector<Node>, greater<Node>> pq;

void Astar(int s) {
    memset(g, 0x3f, sizeof(g));

    g[s] = 0;

    pq.push({h[0], s});

    while (!pq.empty()) {
        int u = pq.top().u;
        pq.pop();
        if (vis[u]) {
            continue;
        }
        vis[u] = 1;
        for (auto ed : edge[u]) {
            int v = ed.v, w = ed.w;
            if (g[v] > g[u] + w) {
                g[v] = g[u] + w;
                pq.push({g[v] + h[v], v});
            }
        }
    }
}

```

7.5. ID-A*

8. 双指针

8.1. 相向指针

8.1.1. 接雨水问题

8.1.2. 最大矩形

8.2. 快慢指针

8.2.1. 环问题

8.3. 滑动窗口

8.3.1. 最小覆盖字串

8.3.2. 无重复字符的最长字串

8.4. 分离指针

8.4.1. 归并排序

9. 进阶数据结构

9.1. 并查集

```
struct DSU {
    vector<size_t> parent, size;
    DSU(size_t n)
        : parent(n),
          size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    size_t find_root(size_t x) {
        return parent[x] == x
            ? x
            : parent[x] = find_root(parent[x]);
    }

    void unite(size_t x, size_t y) {
        x = find_root(x), y = find_root(y);
        if (x == y) {
            return;
        }
        if (size[x] < size[y]) {
            swap(x, y);
        }
        parent[y] = x;
        size[x] += size[y];
    }
};
```

9.2. 线段树

```
/**
 * @note 只有满足结合律的运算才可使用
 * @note 此模板是加法版的 SegTree
 */

// 左闭右闭
struct Interval {
    int left, right; // 所管理 a 的区间
    int root;        // 对应 d 的节点

    int mid(void) const {
        return left + (right - left) / 2;
    }

    int len(void) const {
        return right - left + 1;
    }

    int left_root(void) const {
        return root * 2;
    }

    int right_root(void) const {
        return root * 2 + 1;
    }

    int left_len(void) const {
        return mid() - left + 1;
    }

    int right_len(void) const {
        return right - mid();
    }
};
```

```
};

Interval left_interval(void) const {
    return {left, mid(), root * 2};
}

Interval right_interval(void) const {
    return {mid() + 1, right, root * 2 + 1};
}

bool is_in(int find_left, int find_right) const {
    return find_left <= left && find_right >= right;
}

};

struct SegTree {
    int a[N]; // 原始数据

    int d[4 * N]; // 节点
    int lazy[4 * N]; // 懒标记

    SegTree() {
        fill(d, d + 4 * N, 0);
        fill(lazy, lazy + 4 * N, 0);
    }

    // 由 a 数组建树
    void build(Interval interval) {
        if (interval.left == interval.right) {
            d[interval.root] = a[interval.left];
            return;
        }
        build(interval.left_interval());
        build(interval.right_interval());
        // 节点结合操作
    }
};
```

```

    d[interval.root] = d[interval.left_root()] + d[interval.right_root()];
}

// 分发懒标记
void push_down(Interval interval) {
    // 判断：有懒标记且不是叶子节点
    if (lazy[interval.root] && interval.left < interval.right) {
        // 应用懒标记
        d[interval.left_root()] = d[interval.left_root()] + lazy[interval.root] *
interval.left_len();
        d[interval.right_root()] = d[interval.right_root()] + lazy[interval.root] *
interval.right_len();
        // 下发懒标记
        lazy[interval.left_root()] = lazy[interval.left_root()] +
lazy[interval.root];
        lazy[interval.right_root()] = lazy[interval.right_root()] +
lazy[interval.root];
        // 清空懒标记
        lazy[interval.root] = 0;
    }
}

// 区间查找(find_left==find_right时为单点查找)
int query(int find_left, int find_right, Interval interval) {
    if (interval.is_in(find_left, find_right)) {
        return d[interval.root];
    }
    push_down(interval);

    int mid = interval.mid();
    int sum = 0; // 单位元
    if (find_left <= mid) {
        sum += query(find_left, find_right, interval.left_interval());
    }

```

```

    }
    if (find_right > mid) {
        sum += query(find_left, find_right, interval.right_interval());
    }
    return sum;
}

// 区间更新(update_left==update_right时为单点更新)
void update(int update_left, int update_right, int delta_val, Interval interval) {
    if (interval.is_in(update_left, update_right)) {
        d[interval.root] += delta_val * interval.len(); // 更新节点
        lazy[interval.root] += delta_val; // 更新懒标记
        return;
    }
    push_down(interval);

    int mid = interval.mid();
    if (update_left <= mid) {
        update(update_left, update_right, delta_val, interval.left_interval());
    }
    if (update_right > mid) {
        update(update_left, update_right, delta_val, interval.right_interval());
    }
    // 更新节点
    d[interval.root] = d[interval.left_root()] + d[interval.right_root()];
}

};
SegTree seg;
signed main() {
    Interval init_interval{1, n, 1};
    for (int i = 1; i <= n; ++i) {
        cin >> seg.a[i];
    }
}

```

```

}
seg.build(init_interval);
seg.update(update_left, update_right, 114514, init_interval);
cout << seg.query(find_left, find_right, init_interval);
return 0;
}

```

9.3. 树状数组

```

/**
 * @note 只有满足结合律和差分性质的运算才可使用
 * @note 支持单点修改和区间查询
 * @note i 父亲: i+lowbit(i)
 */
int lowbit(int x) {
    return x & (-x);
}
int a[N];
int n;
int c[N]; // 树状数组节点, c[i] 管理 (i - lowbit(i), i] 这段区间

void build() {
    for (int i = 1; i <= n; ++i) {
        c[i] += a[i];
        int j = i + lowbit(i);
        if (j <= n) {
            c[j] += c[i];
        }
    }
}
// 单点更新
void add(int pos, int val) {

```

```

    for (int i = pos; i <= n; i += lowbit(i)) {
        c[i] += val;
    }
}
// 前缀和
int presum(int pos) {
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i)) {
        res += c[i];
    }
    return res;
}
// 区间查询(左闭右闭)
int query(int left, int right) {
    return presum(right) - presum(left - 1);
}
/**
 * @note 若要求区间修改单点查询, 则改为差分数组
 * @note 若要求区间修改区间查询, 则维护两个数组 d1[], d2[]
 *     d[i]=a[i]-a[i-1]
 *     sum a[i]=sum d[i]*(n-i+1)=sum d[i]*(n+1) + sum d[i]*i
 *     因此 d1[i]=d[i], d2[i]=d[i]*i
 */

```

10. 数论

10.1. 质数

10.1.1. 质数的判定

```
/**
 * @brief 试除法
 */
bool is_prime(int n) {
    if (n < 2) {
        return false;
    }
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}
```

10.1.2. 质数筛法

10.1.2.1. 埃氏筛

```
/**
 * @brief 埃氏筛
 * @details  $O(n \log \log n)$ 
 */
const int N = 10000;
bitset<N> v;
vector<int> primes;
void get_primes_1(int n) {
    v.reset();

    for (int i = 2; i <= n; ++i) {
        if (v[i]) {
            continue;
        }
        primes.push_back(i);
        for (int j = i * i; j <= n; j += i) {
            v[j] = true;
        }
    }
}
```

10.1.2.2. 线性筛

```
/**
 * @brief 线性筛
 * @details  $O(n)$ 
 */
```

```

int u[N];          // 记录最小的质因子
vector<int> primes; // 质数列表
void get_primes_2(int n) {
    memset(u, 0, sizeof(u));
    for (int i = 2; i <= n; ++i) {
        if (u[i] == 0) {
            u[i] = i;
            primes.push_back(i);
        }
        for (auto x : primes) {
            if (x > u[i] || x > n / i) {
                break;
            }
            u[i * x] = x;
        }
    }
}

```

10.1.3. 质因数分解

```

struct Prime {
    int p, c;
};
vector<Prime> p;
void divide(int n) {
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            int cnt = 0;
            while (n % i == 0) {
                n /= i;
                cnt++;
            }
        }
    }
}

```

```

    }
    p.push_back({i, cnt});
}
}
if (n > 1) {
    p.push_back({n, 1});
}
}

```

10.2. gcd

```

/**
 * @name gcd
 */
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}

/**
 * @name lcm
 */
int lcm(int a, int b) {
    return a / gcd(a, b) * b;
}

```

10.3. 因数

11. 动态规划

11.1. 背包

11.1.1. 01 背包

```
/**
 * @brief 01 背包
 */
struct item {
    int w, v;
};
item a[N];
// int dp01[N][W];
int dp01[W];
void beibao01() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v;
    }

    // 法一：二维数组
    // for (int i = 1; i <= n; ++i) {
    //     for (int j = 0; j < a[i].w; ++j) {
    //         dp01[i][j] = dp01[i - 1][j];
    //     }
    //     for (int j = a[i].w; j <= w; ++j) {
    //         dp01[i][j] =
```

```
        //         max(dp01[i - 1][j],
        //         dp01[i - 1][j - a[i].w] + a[i].v);
        //     }
    // }
    // cout << dp01[n][w];

    // 法二：一维数组（注意遍历方向）
    for (int i = 1; i <= n; ++i) {
        for (int j = w; j >= a[i].w; --j) {
            dp01[j] = max(dp01[j], dp01[j - a[i].w] + a[i].v);
        }
    }
    cout << dp01[w];
}
```

11.1.2. 多重背包

```
/**
 * @brief 多重背包
 */
struct item {
    int w, v;
    int c;
};
item a[N];

int dpdc[W];
void beibaodc() {
    int n, w;
    cin >> n >> w;
```

```

for (int i = 1; i <= n; ++i) {
    cin >> a[i].w >> a[i].v >> a[i].c;
}

// 法一: 当成 01 背包, 物品复制 c 份
// for (int i = 1; i <= n; ++i) {
//     // 小剪枝: 如果足够大, 可以直接当成完全背包
//     // begin
//     if (a[i].c * a[i].w >= w) {
//         for (int j = a[i].w; j <= w; ++j) {
//             dpdc[j] =
//                 max(dpdc[j], dpdc[j - a[i].w] +
//                     a[i].v);
//         }
//         continue;
//     }
//     // end
//     while (a[i].c--)
//         for (int j = w; j >= a[i].w; --j) {
//             dpdc[j] =
//                 max(dpdc[j], dpdc[j - a[i].w] +
//                     a[i].v);
//         }
//     }

// 法二: 二进制优化
for (int i = 1; i <= n; ++i) {
    // 小剪枝: 如果足够大, 可以直接当成完全背包
    // begin
    if (a[i].c * a[i].w >= w) {
        for (int j = a[i].w; j <= w; ++j) {
            dpdc[j] = max(dpdc[j], dpdc[j - a[i].w] + a[i].v);
        }
    }
}

```

```

    }
    continue;
}
// end

// 二进制优化
int lei = 0;
for (int k = 1; lei + k <= a[i].c; lei += k, k <= 1) {
    for (int j = w; j >= a[i].w * k; --j) {
        dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * k] + a[i].v * k);
    }
}
int r = a[i].c - lei;
if (r) {
    for (int j = w; j >= a[i].w * r; --j) {
        dpdc[j] = max(dpdc[j], dpdc[j - a[i].w * r] + a[i].v * r);
    }
}

cout << dpdc[w];
}

```

11.1.3. 完全背包

```

/**
 * @brief 完全背包
 */
struct item {
    int w, v;
};

```

```

item a[N];

int dpwq[W];
void beibaowq() {
    int n, w;
    cin >> n >> w;
    for (int i = 1; i <= n; ++i) {
        cin >> a[i].w >> a[i].v;
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = a[i].w; j <= w; ++j) {
            dpwq[j] = max(dpwq[j], dpwq[j - a[i].w] + a[i].v);
        }
    }
    cout << dpwq[w];
}

```

11.1.4. 分组背包

12. 图论

12.1. 存储结构

12.1.1. 链式前向星

```

/**
 * @name 链式前向星
 * @note 适用于有向图
 */
const int N = 1e5 + 5;
int n;
struct Edge {
    int next;    // 下一个坐标
    int to;      // 指向的节点
    int weight;  // 边权重
};
vector<Edge> edge;
vector<int> head(N, -1); // 头节点的初始坐标

void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}

void add_edge(int u, int v, int w) {
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;

    isnt_root[v] = true;
}

```

```

}
bool find_edge(int u, int v) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        if (edge[i].to == v) {
            return true;
        }
    }
    return false;
}

```

/**

* @brief dfs遍历

* @details time O(n)

*/

vector<bool> visited(N);

void dfs(int u) {

if (visited[u]) {

return;

}

visited[u] = true;

for (int i = head[u]; ~i; i = edge[i].next) {

int v = edge[i].to;

int w = edge[i].weight;

dfs(v);

}

}

/**

* @note 寻找根节点->寻找入度为0的节点

*/

/**

* @brief 无向图

* @note edge[i]的反边为 edge[i^1]

*/

void add_edge(int u, int v, int w) {

edge.push_back({head[u], v, w});

head[u] = edge.size() - 1;

edge.push_back({head[v], u, w});

head[v] = edge.size() - 1;

}

12.1.2. 邻接表

const int N = 1e5 + 5;

struct Edge {

int v, w;

};

vector<int> edge[N];

void init_edge() {

for (int i = 0; i < N; ++i) {

edge[i].clear();

}

}

void add_edge(int u, int v, int w) {

edge[u].push_back({v, w});

};

bool find_edge(int u, int v) {

for (auto [_v, w] : edge[u]) {

if (v == _v) {

return true;

```

    }
}
return false;
}
vector<bool> visited(N);
void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;
    for (auto v : edge[u]) {
        dfs(v);
    }
}

```

12.1.3. 邻接矩阵

```

const int N = 1e5 + 5;
int graph[N][N];
void init_edge() {
    memset(graph, 0, sizeof(graph));
}
void add_edge(int u, int v, int w) {
    graph[u][v] = w;
};
bool find_edge(int u, int v) {
    return graph[u][v] != 0;
}
vector<bool> visited(N);
void dfs(int u) {
    if (visited[u]) {
        return;
    }
}

```

```

}
visited[u] = true;
for (int v = 1; v <= n; ++v) {
    if (graph[v]) {
        dfs(v);
    }
}
}

```

12.2. 最短路

12.2.1. Dijkstra

```

/**
 * @brief Dijkstra
 * @details time O(m log n)
 * @details 只允许非负权图
 */
const int N = 1e3;
const int INF = 0x3f3f3f3f;
int n;
struct Edge {
    int v, w;
};
struct Node {
    int u;
    int dis;
    bool operator>(const Node& a) const {
        return dis > a.dis;
    }
}

```

```

};
vector<Edge> edge[N];
int dis[N];
bool vis[N];
priority_queue<Node, vector<Node>, greater<Node>> pq;

void dijkstra(int s) {
    memset(dis, 0x3f, (n + 1) * sizeof(dis[0]));

    dis[s] = 0;
    pq.push({s, 0});
    while (!pq.empty()) {
        int u = pq.top().u;
        pq.pop();
        if (vis[u]) {
            continue;
        }
        vis[u] = true;
        for (auto e : edge[u]) {
            int v = e.v, w = e.w;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                pq.push({v, dis[v]});
            }
        }
    }
}

```

12.2.2. SPFA

```

/**
 * @name SPFA
 * @details time<=O(nm) space O(n)
 * @details 适用于正负边权的图，可以判断无最短路的情况
 */
const int N = 1e3;
const int INF = 0x3f3f3f3f;
struct Edge {
    int v, w;
};
int n;
vector<Edge> edge[N];
int dis[N];
int cnt[N];
bool vis[N];

queue<int> q;
bool SPFA(int s) {
    memset(dis, 0x3f, (n + 1) * (sizeof(dis[0])));

    dis[s] = 0;
    vis[s] = true;
    q.push(s);
    while (!q.empty()) {
        int u = q.front();
        q.pop();

        vis[u] = false;

        for (auto e : edge[u]) {
            int v = e.v, w = e.w;
            if (dis[v] > dis[u] + w) {

```

```

        dis[v] = dis[u] + w;
        cnt[v] = cnt[u] + 1;
        if (cnt[v] >= n) {
            return true;
        }
        if (!vis[v]) {
            q.push(v);
            vis[v] = true;
        }
    }
}
return false;
}

```

12.3. 树

12.3.1. 深度、 2^k 辈父节点

```

/**
 * @brief depth和father
 * @param d[x]:表示x深度
 * @param f[x][k]:表示x的 $2^k$ 辈祖先
 * @details time O(n)
 */

int d[N];
int f[N][LOGN];

```

```

void prework(int root) {
    queue<int> q;
    q.push(root);
    d[root] = 1;
    int t = (int)(log(n) / log(2)) + 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            if (d[v]) {
                continue;
            }
            q.push(v);

            d[v] = d[u] + 1;

            f[v][0] = u;
            for (int j = 1; j <= t; ++j) {
                f[v][j] = f[f[v][j - 1]][j - 1];
            }
        }
    }
}

```

12.3.2. 树的中心

```

/**

```

```

* @brief 树的中心
* @property 1.在直径上
* @property 2.中心不唯一，但最多有 2 个，且两个中心相邻
* @property 3.所有点到最远点的路径一定交汇于树的中心
* @property 4.当中心为根节点时，到达直径两端的两条链一定分别是最长链和次长链
* @property 5.树的中心到其他节点距离不超过直径的一半
* @details 求法：寻找一个点 x，使其作为根节点时的最长链长度最短
* @details time O(n)
*/
int d1[N]; // 以 1 为节点时 x 子树的最长链
int d2[N]; // 以 1 为节点时 x 子树的次长链
int up[N]; // x 子树外的最长链，必然经过 x 的父节点
// 如果找到点 x 使得 max(d1[x], up[x]) 最小，则 x 即为树的中心
void dfs_down(int u, int father) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == father) {
            continue;
        }
        dfs_down(v, u);

        if (d1[v] + w > d1[u]) {
            d2[u] = d1[u];
            d1[u] = d1[v] + w;
        }
        else if (d1[v] + w > d2[u]) {
            d2[u] = d1[v] + w;
        }
    }
}
void dfs_up(int u, int father) {

```

```

    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        if (v == father) {
            continue;
        }
        up[v] = up[u] + w;
        if (d1[v] + w != d1[u]) {
            up[v] = max(up[v], d1[u] + w);
        }
        else {
            up[v] = max(up[v], d2[u] + w);
        }
        dfs_up(v, u);
    }
}
int get_center() {
    dfs_down(1, 0);
    dfs_up(1, 0);
    int minlen = INT_MAX;
    int ans = 0;
    int ans2 = 0;
    for (int i = 1; i <= n; ++i) {
        // 其中一个中心
        if (max(d1[i], up[i]) < minlen) {
            minlen = max(d1[i], up[i]);
            ans = i;
            ans2 = 0;
        }
        // 另一个中心
        else if (max(d1[i], up[i]) == minlen) {
            ans2 = i;

```

```

    }
}
return ans;
}

```

12.3.3. LCA

```

/**
 * @brief 树的直径
 * @details 树形 dp
 * @param d[x]: 以 x 节点为根节点，向下延申可形成的最长距离
 * @param ans: 经过 x 节点形成的最长直径
 */
int d[N];
int ans;
void dp(int u, int fa) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;

        if (v == fa) {
            continue;
        }
        dp(v, u);

        // 这里的 d[u] 为 1-(i-1) 内的最大向下距离
        ans = max(ans, d[u] + d[v] + w);
        d[u] = max(d[u], d[v] + w);
    }
}

```

```

/**
 * @brief LCA
 * @details 树上倍增
 * @details 预处理  $O(n \log n)$ ，询问  $O(\log n)$ 
 * @param f[x][k]: 表示 x 的  $2^k$  辈祖先
 * @param d[x]: 表示 x 深度
 */

int f[N][LOGN];
int d[N];

void prework(int root) {
    queue<int> q;
    q.push(root);
    d[root] = 1;
    int t = (int)(log(n) / log(2)) + 1;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            if (d[v]) {
                continue;
            }
            q.push(v);

            d[v] = d[u] + 1;

            f[v][0] = u;

```

```

        for (int j = 1; j <= t; ++j) {
            f[v][j] = f[f[v][j - 1]][j - 1];
        }
    }
}

int lca(int x, int y) {
    if (d[x] > d[y]) {
        swap(x, y);
    }
    int t = (int)(log(n) / log(2)) + 1;

    for (int i = t; i >= 0; --i) {
        if (d[f[y][i]] >= d[x]) {
            y = f[y][i];
        }
    }
    if (x == y) {
        return x;
    }
    for (int i = t; i >= 0; --i) {
        if (f[x][i] != f[y][i]) {
            x = f[x][i];
            y = f[y][i];
        }
    }
    return f[x][0];
}

```

12.3.4. 树的重心

```

int size[N];
int now_min; // 去掉重心后的最大子树大小
int ans;

void dfs(int u) {
    if (visited[u]) {
        return;
    }
    visited[u] = true;

    size[u] = 1;

    int max_part = 0; // 表示删掉 x 后分成的最大子树的大小

    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        dfs(v);

        size[u] += size[v];

        max_part = max(max_part, size[v]);
    }
    max_part = max(max_part, n - size[u]);
    if (max_part < now_min) {
        now_min = max_part;
        ans = u;
    }
}

```

12.4. 生成树

12.4.1. 最小生成树

12.4.1.1. Kruskal 算法

```
/**
 * @brief Kruskal算法
 * @details time O(m log m)
 */
struct DSU {
    vector<size_t> parent, size;
    DSU(size_t n)
        : parent(n),
          size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    size_t find_root(size_t x) {
        return parent[x] == x ? x : parent[x] = find_root(parent[x]);
    }

    void unite(size_t x, size_t y) {
        x = find_root(x), y = find_root(y);
        if (x == y) {
            return;
        }
        if (size[x] < size[y]) {
            swap(x, y);
        }
        parent[y] = x;
        size[x] += size[y];
    }
};
```

```
struct Temp_edge {
    int u, v, w;
};

bool operator<(const Temp_edge& a, const Temp_edge& b) {
    return a.w < b.w;
}

void kruskal(vector<Temp_edge>& lists) {
    int n = lists.size();
    sort(lists.begin(), lists.end());

    DSU dsu(n);
    for (auto [u, v, w] : lists) {
        int x = dsu.find_root(u);
        int y = dsu.find_root(v);
        if (x == y) {
            continue;
        }
        dsu.unite(x, y);

        // add_edge(u, v, w);
    }
}
```

12.4.1.2. Prim 算法

```
/**
 * @brief Prim算法
 * @details O(m log n)
 */
```

```

const int N = 1e5 + 5;
int dis[N]; // 到当前生成树的最小距离
// pii=<dis,u>
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
vector<bool> visited;
int prim(int root) {
    visited.resize(n, false);
    memset(dis, 0x3f, sizeof(dis));
    dis[root] = 0;
    q.push({0, root});
    int cnt = 0;
    while (!q.empty()) {

        auto [d, u] = q.top();
        q.pop();

        if (visited[u]) {
            continue;
        }
        visited[u] = true;

        ++cnt;

        // do_something(father,u,d);

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;

            if (w < dis[v]) {

```

```

                dis[v] = w;
                q.push({w, v});
            }
        }

        if (cnt >= n) {
            break;
        }
    }
    if (cnt < n) {
        // 不连通
        return -1;
    }
    return 0;
}

```

12.5. 二分图

12.5.1. 染色判定法

```

/**
 * @brief 二分图的染色判定法
 */
int color[N];
bool dfs_color(int u, int c) {
    color[u] = c;
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;

```

```

    if (color[v] == 0) {
        if (!dfs_color(v, 3 - c)) {
            return false;
        }
    }
    else if (color[v] == c) {
        return false;
    }
}
return true;
}
bool judge() {
    for (int i = 1; i <= n; ++i) {
        if (color[i] == 0) {
            if (!dfs_color(i, 1)) {
                return false;
            }
        }
    }
    return true;
}
}

```

12.5.2. 最大匹配数(匈牙利算法)

```

/**
 * @brief 二分图的最大匹配
 * @brief 匈牙利算法
 * @details time O(n m)
 * @return 最大匹配数
 */
int match[N];

```

```

int visited[N];
bool dfs(int u) {
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;

        if (!visited[v]) {
            visited[v] = true;
            if (!match[v] || dfs(match[v])) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int Hungarian() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
        memset(visited, 0, sizeof(visited));
        if (dfs(i)) {
            ans++;
        }
    }
    return ans;
}

```

12.6. 网络流

12.6.1. 最大流

12.6.1.1. Edmonds-Karp 算法

```
/**
 * @name Edmonds-Karp 算法
 * @details 时间复杂度  $O(V * E^2)$ 
 */
struct Edge {
    int next;
    int to;
    int weight; // capacity
};
vector<Edge> edge;

vector<int> head(N, -1);

void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}

void add_edge(int u, int v, int w) {
    // 正图
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;
    // 反图
    edge.push_back({head[v], u, 0});
    head[v] = edge.size() - 1;
}

// pre[v] 记录到达 v 的边的下标
int pre[N];
// minflow[v] 是 s 到 v 的最小剩余容量
int minflow[N];
```

```
bool bfs(int s, int t) {
    fill(pre, pre + N, -1);
    memset(minflow, 0, sizeof(minflow));
    queue<int> q;
    q.push(s);
    minflow[s] = INF;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;

            // 未访问或有剩余容量
            if (pre[v] == -1 && w > 0) {
                pre[v] = i;
                minflow[v] = min(minflow[u], w);
                q.push(v);
            }
        }
    }

    return pre[t] != -1;
}

int max_flow(int s, int t) {
    int flow = 0;
    while (bfs(s, t)) { // bfs 找路
        // 从 t 开始遍历沿着反图走到 s
        for (int v = t; v != s; v = pre[v]) {
            int i = pre[v];
            edge[i].weight -= minflow[t];
        }
    }
}
```

```

        edge[i ^ 1].weight += minflow[t];
        v = edge[i ^ 1].to;
    }
    flow += minflow[t];
}
return flow;
}

```

12.6.1.2. Dinic 算法

```

/**
 * @name Dinic算法
 * @details 时间复杂度  $O(V^2 * E)$ 
 */
struct Edge {
    int next;
    int to;
    int weight;
};
vector<Edge> edge;
vector<int> head(N, -1);
void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}
void add_edge(int u, int v, int w) {
    // 正图
    edge.push_back({head[u], v, w});
    head[u] = edge.size() - 1;
    // 反图
    edge.push_back({head[v], u, 0});
    head[v] = edge.size() - 1;
}

```

```

}

vector<int> level(N, -1); // 分层图
vector<int> cur(N, -1);   // 当前弧优化, cur[u]表示遍历到哪条边
// bfs 找路, 建立分层图, 让 dfs 只允许走最短路
bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);

    queue<int> q;
    q.push(s);
    level[s] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = head[u]; ~i; i = edge[i].next) {
            int v = edge[i].to;
            int w = edge[i].weight;
            // 如果没有经过 v 或有剩余容量
            if (level[v] == -1 && w > 0) {
                level[v] = level[u] + 1;
                q.push(v);
            }
        }
    }

    return level[t] != -1;
}

// 按分层图找 u 到 t 的增广路
int dfs(int u, int t, int flow) {
    if (u == t) {
        return flow; // 找到增广路即返回
    }
}

```

```

    }
    for (int& i = cur[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        int w = edge[i].weight;
        // 只能走到下一层且有剩余容量的点
        if (level[v] == level[u] + 1 && w > 0) {
            int d = dfs(v, t, min(flow, w));
            // 如果之后的v到t有最大流量d
            if (d > 0) {
                edge[i].weight -= d;
                edge[i ^ 1].weight += d; // 反向边
                return d;
            }
        }
    }
    return 0;
}

int max_flow(int s, int t) {
    int ans = 0;
    while (bfs(s, t)) {
        copy(head.begin(), head.end(), cur.begin());
        int f = 0;
        while ((f = dfs(s, t, INF)) > 0) {
            ans += f;
        }
    }
    return ans;
}

```

12.7. 连通性

12.7.1. 强连通分量 SCC

12.7.1.1. Tarjan 算法

```

/**
 * @name Tarjan算法
 * @details O(V+E)
 */
struct Edge {
    int next;
    int to;
    int weight;
};

vector<Edge> edge;
vector<int> head(N, -1);

int dfn[N]; // dfs 搜索时, u 被搜索到的 timestamp。从根节点开始的一个路径上 dfn[u]单调递增
int low[N]; // 从 u 出发, dfs 子树中, 能追溯到的最早的仍在栈里的节点的 dfn。从根节点开始的一个路径上 low[u]单调不增
int timestamp; // 全局时间戳

int SCC_count; // SCC 个数
int SCC_id[N]; // 节点 i 所在的 SCC 编号
int SCC_size[N]; // 编号为 i 的 SCC 的大小

bool in_stack[N]; // 节点 u 是否在栈中
stack<int> s; // 栈

void tarjan(int u) {
    low[u] = dfn[u] = ++timestamp;

```

```

s.push(u);
in_stack[u] = true;
// 遍历一次u树
for (int i = head[u]; ~i; i = edge[i].next) {
    int v = edge[i].to;

    // 如果 v 没有被搜索过
    if (!dfn[v]) {
        tarjan(v);
        low[u] = min(low[u], low[v]);
    }
    // 如果 v 被搜索过并且在栈中, 则说明找到了返祖边
    else if (in_stack[v]) {
        low[u] = min(low[u], dfn[v]);
    }
}
// 找到 SCC 的根节点
if (dfn[u] == low[u]) {
    ++SCC_count;
    int v;
    // 弹出我和我的子树(即找到了一组 SCC)
    do {
        v = s.top();
        s.pop();
        SCC_id[v] = SCC_count;
        ++SCC_size[SCC_count];
        in_stack[v] = false;
    } while (u != v);
}
}
void solve() {
    for (int i = 1; i <= n; ++i) {

```

```

        if (!dfn[i]) {
            tarjan(i);
        }
    }
}

```

12.7.1.2. Kosaraju 算法

```

/**
 * @brief Kosaraju 算法
 * @details O(V+E)
 */
vector<int> edge[N]; // 正向图
vector<bool> vis;
vector<int> edge2[N]; // 反向图
vector<bool> vis2;

vector<int> post_order; // 后序遍历节点表

int SCC_count;
int SCC_id[N];
int SCC_size[N];

// 正图上后序遍历, 得到拓扑序
void dfs1(int u) {
    vis[u] = true;
    for (auto v : edge[u]) {
        if (!vis[v]) {
            dfs1(v);
        }
    }
    post_order.push_back(u);
}

```

```

}
// 反图上扩散
void dfs2(int u) {
    SCC_id[u] = SCC_count;
    ++SCC_size[SCC_count];
    for (auto v : edge2[u]) {
        if (!SCC_id[v]) {
            dfs2(v);
        }
    }
}

void kosaraju() {
    // 第一次 dfs 进行后序遍历
    for (int i = 1; i <= n; ++i) {
        if (!vis[i]) {
            dfs1(i);
        }
    }
    // 对后序遍历的表进行反向遍历
    for (auto it = post_order.rbegin(); it != post_order.rend(); ++it) {
        int u = *it;
        // 如果还没开始标 SCC 的序号
        if (!SCC_id[u]) {
            ++SCC_count;
            dfs2(u);
        }
    }
}
}

```

12.7.2. 双连通分量 BCC

12.7.2.1. e-BCC

```

/**
 * @brief tarjan 算法
 * @details 无向图，边-双连通即为任意去掉一条边仍然互相连通
 * @details O(E+V)
 */
struct Edge {
    int next;
    int to;
};
vector<Edge> edge;
vector<int> head(N, -1);

void add_edge(int u, int v) {
    edge.push_back({head[u], v});
    head[u] = edge.size() - 1;
    edge.push_back({head[v], u});
    head[v] = edge.size() - 1;
}

int dfn[N];
int low[N];
int timestamp;

bool is_bridge[N]; // 边 i 是否为桥

int BCC_count; // e-BCC 个数
int BCC_id[N]; // 节点 u 所在 BCC 编号
int BCC_size[N]; // 编号为 i 的 BCC 大小

void tarjan(int u, int in_edge) { // 节点 u，入节点 u 的边

```

```

low[u] = dfn[u] = ++timestamp;
for (int i = head[u]; ~i; i = edge[i].next) {
    // 不能回去
    if (i == (in_edge ^ 1)) {
        continue;
    }

    int v = edge[i].to;

    // v 没有被搜索过->树边
    if (!dfn[v]) {
        tarjan(v, i);
        low[u] = min(low[u], low[v]);
        if (low[v] > dfn[u]) {
            is_bridge[i] = is_bridge[i ^ 1] = true;
        }
    }
    // v 被搜索过->回边
    else {
        low[u] = min(low[u], dfn[v]);
    }
}
}

```

```

void dfs_BCC(int u) {
    BCC_id[u] = BCC_count;
    ++BCC_size[BCC_count];

    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;
        // 是桥或者已标号
        if (is_bridge[i] || BCC_id[v]) {

```

```

        continue;
    }
    dfs_BCC(v);
}
}

void solve() {
    for (int i = 1; i <= n; ++i) {
        if (!dfn[i]) {
            tarjan(i, -1);
        }
    }
    for (int i = 1; i <= n; ++i) {
        if (!BCC_id[i]) {
            ++BCC_count;
            dfs_BCC(i);
        }
    }
}

```

12.7.2.2. v-BCC

```

/**
 * @note 两个 v-BCC 最多有一个公共点，其为割点
 * @note v-BCC 在 dfs 搜索树中 dfn 最小的点 一定是割点或树根
 *      1. 当这个点为割点时，一定是点双连通分量的根
 *      2. 当这个点为树根时，
 *          a. 有两个以上的子树，那他一定是割点
 *          b. 只有一个子树，那他是一个点双连通分量的根
 *          c. 没有子树，那他自己成为一个点双连通分量
 * @details O(V+E)
 */

```

```

struct Edge {
    int next;
    int to;
};
vector<Edge> edge;
vector<int> head(N, -1);
void init_edge() {
    edge.clear();
    fill(head.begin(), head.end(), -1);
}
void add_edge(int u, int v) {
    edge.push_back({head[u], v});
    head[u] = edge.size() - 1;
    edge.push_back({head[v], u});
    head[v] = edge.size() - 1;
}

int dfn[N];
int low[N];
int timestamp;

bool is_cut[N];

int root; // 此时 dfs 搜索树是以谁为 root
stack<int> s;

int BCC_count; // v-BCC 个数
vector<int> BCC[N]; // BCC[i] 表示第 i 个 v-BCC, 注: 有些割点是重复的

void tarjan(int u) {
    dfn[u] = low[u] = ++timestamp;
    s.push(u);

```

```

    int child_num = 0; // 记录满足 low[v] >= dfn[u] 的子节点个数
    for (int i = head[u]; ~i; i = edge[i].next) {
        int v = edge[i].to;

        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
            // 判据: 无法回溯到更早的地方, 则 u 一定是一个割点或根节点
            if (low[v] >= dfn[u]) {
                // 如果不是树根, 一定为割点; 如果有两个以上的子树, 则一定为割点
                if (++child_num > 1 || u != root) {
                    is_cut[u] = true;
                }

                // 说明发现 v-BCC
                ++BCC_count;
                int x;
                do {
                    x = s.top();
                    s.pop();
                    BCC[BCC_count].push_back(x);
                } while (x != v);
                BCC[BCC_count].push_back(u);
            }
        }
        else {
            low[u] = min(low[u], dfn[v]);
        }
    }
}

void solve() {

```

```

for (int i = 1; i <= n; ++i) {
    if (!dfn[i]) {
        if (~head[i]) {
            while (!s.empty()) {
                s.pop();
            }
            root = i;
            tarjan(i);
        }
        else {
            dfn[i] = low[i] = ++timestamp;
            ++BCC_count;
            BCC[BCC_count].push_back(i);
        }
    }
}
}

```

13. 字符串

13.1. KMP 模式匹配

```

string s;
string k;
vector<int> kmp;
void set_kmp(void) {
    kmp[0] = 0;
    for (int i = 1; i <= k.size(); ++i) {
        int j = kmp[i - 1];
        while (j > 0 && k[i] != k[j]) {
            j = kmp[j - 1];
        }
        if (k[i] == k[j]) {
            ++j;
        }
        kmp[i] = j;
    }
}
int main() {
    cin >> s >> k;
    kmp.resize(k.size());
    set_kmp();
    for (int i = 0, j = 0; i < s.size(); ++i) {
        while (j > 0 && s[i] != k[j]) {
            j = kmp[j - 1];
        }
        if (s[i] == k[j]) {
            ++j;
        }
    }
}

```

```

    }
    if (j == k.size()) {
        cout << i - k.size() + 2 << endl;
        j = kmp[j - 1];
    }
}
for (int i = 0; i < kmp.size(); ++i) {
    cout << kmp[i] << " ";
}
return 0;
}

```

```

bool find(const char* s, int len) {
    int ptr = 0;
    for (int i = 0; i < len; ++i) {
        int c = s[i] - 'a';
        if (!next[ptr][c]) {
            return false;
        }
        ptr = next[ptr][c];
    }
    return exist[ptr];
}
};

```

13.2. Trie 字典树

```

struct trie {
    int next[100000][26];
    int cnt;
    bool exist[100000];

    void insert(const char* s, int len) {
        int ptr = 0;
        for (int i = 0; i < len; ++i) {
            int c = s[i] - 'a';
            if (!next[ptr][c]) {
                ++cnt;
                next[ptr][c] = cnt;
            }
            ptr = next[ptr][c];
        }
        exist[ptr] = true;
    }
}

```

13.3. SA

```

string s;
int sa[N << 1];
int rk[N << 1];

// O(n^2 log n)
void get_sa1(void) {
    iota(sa, sa + s.size(), 0);
    sort(sa, sa + s.size(), [&](int i, int j) {
        return s.substr(i) < s.substr(j);
    });
}

void get_rk_from_sa(void) {
    for (int i = 0; i < s.size(); ++i) {
        rk[sa[i]] = i;
    }
}

```

```

// O(n log^2 n)
int oldrk[N << 1];
void get_sa2(void) {
    // 第一层排序
    for (int i = 0; i < s.size(); ++i) {
        sa[i] = i;
        rk[i] = s[i];
    }
    // 二层及以上排序
    for (int w = 1; w < s.size(); w <= 1) {
        sort(sa, sa + s.size(), [&](int i, int j) {
            // 这里需要两倍空间, 否则就得老老实实防止越界
            return rk[i] == rk[j] ? rk[i + w] < rk[j + w] : rk[i] < rk[j];
        });
        for (int i = 0; i < s.size(); ++i) {
            oldrk[i] = rk[i];
        }
        // 双指针去重
        rk[sa[0]] = 0;
        for (int i = 1, j = 0; i < s.size(); ++i) {
            if (oldrk[sa[i]] == oldrk[sa[i - 1]] && oldrk[sa[i] + w] == oldrk[sa[i - 1] + w]) {
                rk[sa[i]] = j;
            }
            else {
                rk[sa[i]] = ++j;
            }
        }
    }
}

```

13.4. SAM

```

struct state {
    int len;
    int link;
    map<char, int> next;
};

const int N = 1000000;
state st[N * 2];
int sz, last;
void sam_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}
void sam_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    }
    else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        }
    }
}

```

```

        else {
            int clone      = sz++;
            st[clone].len  = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p              = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

void sam_build(string s) {
    sam_init();
    for (auto x : s) {
        sam_extend(x);
    }
}

```

14. 随机化

14.1. 随机函数

14.1.1. 生成函数

```

/**
 * @brief 随机数生成函数
 */
// 基于硬件的均匀分布随机数生成器
static random_device rd;
// 32 位梅森缠绕器(@ret u32)
// 可替换为 64 位 mt19937_64
static mt19937 gen(rd());
// uniform_int_distribution为整数, 左闭右闭[]
// 浮点数用 uniform_real_distribution, 左闭右开[]
static uniform_int_distribution<int> dist(1, 100);
int get_rand() {
    return dist(gen);
}

```

14.1.2. 随机打乱

```

/**
 * @brief 随机打乱
 */
void algo_shuffle(vector<int>& v) {
    shuffle(v.begin(), v.end(), gen); // 此处 gen 可换为一个随机数发生函数
}

```

```
}
```

14.1.3. 随机挑选

```
/**
 * @brief 随机挑选
 */
vector<int> algo_sample(const vector<int>& v, int n) {
    vector<int> ret;
    sample(v.begin(), v.end(), ret.end(), n, gen);
    return ret;
}
```

14.2. 爬山与退火

15. 离线算法

15.1. 整体二分

15.1.1. 区间第 k 小

```
/**
 * @brief 整体二分+树状数组
 * @details 静态区间第 k 小
 * @details 给定长度为 n 的序列 a, m 次询问区间 [l,r] 内第 k 小值。
           输入: 第一行 n,m; 第二行 a_1,...,a_n; 接下来 m 行每行 l,r,k。
           数据: 1 ≤ n,m ≤ 2e5, 0 ≤ a_i ≤ 1e9, 1 ≤ l ≤ r ≤ n, 1 ≤ k ≤ r-l+1。
 */
#include...
// #define int long long

const int N = 2e5 + 5;

int n;
//=====BIT=====
int lowbit(int x) {
    return x & (-x);
}
// int b[N]; // 原数组第 i 个位置的值, 是否小于等于当前的 mid
int c[N]; // 树状数组节点, c[i] 管理 (i - lowbit(i), i] 这段区间
// 单点更新
void add(int pos, int val) {
    for (int i = pos; i <= n; i += lowbit(i)) {
        c[i] += val;
    }
}
```

```

    }
}
// 前缀和
int presum(int pos) {
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i)) {
        res += c[i];
    }
    return res;
}
// 区间查询(左闭右闭)
int query(int left, int right) {
    return presum(right) - presum(left - 1);
}

//-----整体二分-----

int a[N];          // 原数据-对按照原序的离散化的数据
pair<int, int> b[N]; // 排序离散后的数据<value,idx>
int toRaw[N];      // rank 下标对应的值

int m;

int ans[N];

struct Event {
    enum EventType {
        ADD,
        QUERY
    } event_type;

    int l, r, k; // query 的内容

```

```

    int idx;      // 在对应数组中的 idx
};
vector<Event> event;
vector<Event> leftE, rightE;

// el,er: Query 的 idx
// L,R: 值域
void solve(int el, int er, int L, int R) {
    if (el > er) {
        return;
    }

    if (L == R) {
        for (int i = el; i <= er; ++i) {
            if (event[i].event_type == Event::QUERY) {
                ans[event[i].idx] = L;
            }
        }
        return;
    }

    int MID = (L + R) >> 1;

    leftE.clear(), rightE.clear();
    for (int i = el; i <= er; ++i) {
        if (event[i].event_type == Event::ADD) {
            int idx = event[i].idx;
            if (a[idx] <= MID) {
                add(idx, 1);
                leftE.push_back(event[i]);
            }
            else {

```

```

        rightE.push_back(event[i]);
    }
}

for (int i = el; i <= er; ++i) {
    if (event[i].event_type == Event::QUERY) {
        auto& e = event[i];
        int cnt = query(e.l, e.r);
        if (cnt >= e.k) {
            leftE.push_back(event[i]);
        }
        else {
            e.k -= cnt;
            rightE.push_back(event[i]);
        }
    }
}

// 只需要清空左半边的 event 即可, 因为右边的插入事件本来就不会被 add 进入
for (const auto& e : leftE) {
    if (e.event_type == Event::ADD) {
        int idx = e.idx;
        if (a[idx] <= MID) {
            add(idx, -1);
        }
    }
}

int cnt = 0;
for (const auto& x : leftE) {
    event[el + cnt] = x;
    cnt++;
}

```

```

    }
    int mid = el + cnt;
    for (const auto& x : rightE) {
        event[el + cnt] = x;
        cnt++;
    }
    solve(el, mid - 1, L, MID);
    solve(mid, er, MID + 1, R);
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;

    for (int i = 1; i <= n; ++i) {
        cin >> a[i];
        b[i] = {a[i], i};
    }

    //=====离散化=====
    sort(b + 1, b + 1 + n);
    int rank = 0;
    int last_value = -1;
    for (int i = 1; i <= n; ++i) {
        auto [v, idx] = b[i];
        if (v != last_value) {
            ++rank;
            last_value = v;
        }
        a[idx] = rank;
        toRaw[rank] = v;
    }
}

```

```

}
//-----事件化-----
for (int i = 1; i <= n; ++i) {
    Event e;
    e.event_type = Event::ADD;
    e.idx = i;
    event.push_back(e);
}
for (int i = 1; i <= m; ++i) {
    Event e;
    e.event_type = Event::QUERY;
    cin >> e.l >> e.r >> e.k;
    e.idx = i;
    event.push_back(e);
}

solve(0, event.size() - 1, 1, rank);

for (int i = 1; i <= m; ++i) {
    cout << toRaw[ans[i]] << endl;
}

return 0;
}

```

15.1.2. 单点修改区间第 k 小

```

/**
 * @brief 整体二分+树状数组
 * @details 单点修改静态区间第 k 小
 * @details 给定长度为 n 的序列 a, m 次操作: Q l r k 查询区间 [l,r] 内第 k 小值; C x y 将 a_x 改为 y。
          输入: 第一行 n,m; 第二行 a_1,...,a_n; 接下来 m 行每行一个操作 Q l r k 或 C x y。

```

数据: $1 \leq n, m \leq 1e5, 1 \leq l \leq r \leq n, 1 \leq k \leq r-l+1, 1 \leq x \leq n, 0 \leq a_i, y \leq 1e9$ 。

* @note 也可使用离散化, 但是不离散最多 30 层, 不离散 18 层, 只差常数级

```

*/
INCLUDE...
using namespace std;
// #define int long long

const int N = 2e5 + 5;

int n;
//-----BIT-----
int lowbit(int x) {
    return x & (-x);
}
// int b[N]; // 原数组第 i 个位置的值, 是否小于等于当前的 mid
int c[N]; // 树状数组节点, c[i] 管理 (i - lowbit(i), i] 这段区间
// 单点更新
void add(int pos, int val) {
    for (int i = pos; i <= n; i += lowbit(i)) {
        c[i] += val;
    }
}
// 前缀和
int presum(int pos) {
    int res = 0;
    for (int i = pos; i > 0; i -= lowbit(i)) {
        res += c[i];
    }
    return res;
}
// 区间查询(左闭右闭)
int query(int left, int right) {

```

```

    return presum(right) - presum(left - 1);
}

//-----整体二分-----

int a[N]; // 原数据

int m;

int ans[N];

struct Event {
    enum EventType {
        ADD,
        DELETE,
        QUERY
    } event_type;

    int l, r, k; // query 的内容
    int idx;    // 在对应数组中的 idx
};

vector<Event> event;
vector<Event> leftE, rightE;

// el,er: Query 的 idx
// L,R: 值域
void solve(int el, int er, int L, int R) {
    if (el > er) {
        return;
    }

    if (L == R) {

```

```

        for (int i = el; i <= er; ++i) {
            if (event[i].event_type == Event::QUERY) {
                ans[event[i].idx] = L;
            }
        }
        return;
    }

    int MID = (L + R) >> 1;

    leftE.clear(), rightE.clear();
    for (int i = el; i <= er; ++i) {
        if (event[i].event_type == Event::QUERY) {
            auto& e = event[i];
            int cnt = query(e.l, e.r);
            if (cnt >= e.k) {
                leftE.push_back(event[i]);
            }
            else {
                e.k -= cnt;
                rightE.push_back(event[i]);
            }
        }
        else {
            auto& e = event[i];
            if (e.k <= MID) {
                add(e.idx, e.event_type == Event::ADD ? 1 : -1);
                leftE.push_back(e);
            }
            else {
                rightE.push_back(e);
            }
        }
    }

```

```

    }
}

// 只需要清空左半边的 event 即可，因为右边的事件本来就不会被 add 进入
for (const auto& e : leftE) {
    if (e.event_type == Event::ADD || e.event_type == Event::DELETE) {
        int idx = e.idx;
        if (e.k <= MID) {
            add(idx, e.event_type == Event::ADD ? -1 : 1);
        }
    }
}

int cnt = 0;
for (const auto& x : leftE) {
    event[el + cnt] = x;
    cnt++;
}

int mid = el + cnt;
for (const auto& x : rightE) {
    event[el + cnt] = x;
    cnt++;
}

solve(el, mid - 1, L, MID);
solve(mid, er, MID + 1, R);
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;

```

```

for (int i = 1; i <= n; ++i) {
    cin >> a[i];
}

//=====事件化=====
for (int i = 1; i <= n; ++i) {
    Event e;
    e.event_type = Event::ADD;
    e.idx = i;
    e.k = a[i];
    event.push_back(e);
}

for (int i = 1; i <= m; ++i) {
    char c;
    cin >> c;
    if (c == 'C') {
        Event e;
        e.event_type = Event::DELETE;
        cin >> e.idx;
        e.k = a[e.idx];
        event.push_back(e);
        e.event_type = Event::ADD;
        cin >> e.k;
        a[e.idx] = e.k;

        event.push_back(e);
    }
    else {
        Event e;
        e.event_type = Event::QUERY;
        cin >> e.l >> e.r >> e.k;
        e.idx = i;
    }
}

```

```

        event.push_back(e);
    }
}

fill(ans + 1, ans + m + 1, -1);

solve(0, event.size() - 1, 0, 1e9 + 1);

for (int i = 1; i <= m; ++i) {
    if (ans[i] != -1) {
        cout << ans[i] << endl;
    }
}

return 0;
}

```

15.2. 莫队算法

15.2.1. 普通莫队

```

/**
 * @brief 普通莫队
 */
int n, m; // n是点数, m是询问数
int block_size;

void init() {
    block_size = sqrt(n); // 有时候 n/sqrt(m)会更快
}

int block(int x) {
    return x / block_size;
}

```

```

}

struct Query {
    int l, r;
    int id;

    int len() const {
        return (r - l + 1);
    }

    // 奇偶化排序
    bool operator<(const Query& _) const {
        if (block(l) != block(_.l)) {
            return l < _.l;
        }
        return (block(l) & 1) ? r < _.r : r > _.r;
    }
} q[N];

int ans[N];

int c[N]; // 数据

void add(int x) {
}

void del(int x) {
}

void solve() {
    sort(q + 1, q + m + 1);

    int curL = 1, curR = 0; // 左闭右闭
    for (int i = 1; i <= m; ++i) {

```

```

// 注意 add 是先自增自减, del 是后自增自减
// 最好不要改变这里四个 while 的顺序
// 可用的 while 顺序: 先扩大后缩小 (当然不只有这四种)
while (curL > q[i].l)
    add(--curL);
while (curR < q[i].r)
    add(++curR);
while (curL < q[i].l)
    del(curL++);
while (curR > q[i].r)
    del(curR--);

ans[q[i].id] = ;
}
}

```

15.2.2. 单点修改的莫队

```

/**
 * @brief 单点修改的莫队
 * @note 只需要增加一个维度—修改维度
 * @note operator<的处理为: 再在后面加一个 return t<_.t
 * @note 新加的两个 while 应放到其他四个 while 前面或后面
 */

```